

Data Preparation & Modeling Initiation

MACHINE LEARNING APPLIED PROJECT
PRESENTED BY: SOYOSA



PROJECT PLAN

01 Introduction

02 Problem Statement

03 Existing Solutions

04 Proposed Solution

05 Business Objectives &
Data Science Objectives

06 Methodology

07 **Phase 1:** Data Collection,
Understanding & Preparation

08 **Phase 2:** Modeling

09 Tools and Technologies

10 Conclusion



Introduction

In today's digital era, online learning platforms play a crucial role in education by providing flexible and accessible learning experiences.

These platforms generate large amounts of data related to student behavior, engagement, and performance. When properly analyzed, this data can significantly improve the quality of learning and decision-making processes.

Leveraging data and Machine Learning has become essential to enhance learning outcomes, personalize education, and optimize overall platform performance.

Problem Statement

Despite the availability of large volumes of educational data, many online learning platforms do not fully exploit this information.

This results in several challenges, such as difficulties in monitoring student progress, identifying learners at risk of dropout, and evaluating the effectiveness of educational content.

How can data and Machine Learning be used to better understand student behavior and improve learning outcomes in online education systems?

Existing Solutions

AMIDEAST

- Traditional institutional English education
- Certified programs with qualified tutors
- Structured curriculum with certification
- Limited digital tools and personalization



DUOLINGO

- Gamified and self-paced learning platform
- Interactive lessons with reward system
- Instant automated feedback
- Accessible worldwide (freemium model)



Critique of Existing Solutions

AMIDEAST

- Limited use of digital and data-driven tools
- Lack of personalization
- No integrated event or community management
- Basic administrative features



DUOLINGO

- No human tutors or personalized guidance
- Lack of institutional and administrative tools
- No scheduling or structured learning path
- Limited support for advanced or professional learning



Proposed Solution

A data-driven learning platform combining traditional education and digital innovation.

Key Features:

Personalized Learning : adaptive content

Dropout Prediction : identify at-risk students

Performance Monitoring : real-time tracking

Content Recommendation : suggest optimal courses

Integrated Features : tutors, scheduling, management

Business Objectives & Data Science Objectives

Objective 1 – Improving Student Learning Outcomes

Business Perspective: Anticipates student progress levels

The DS Goal: Predictive student next levels

Objective 2 – Reducing Student Dropout & Inactivity

Business Perspective: Identify at-risk students before they disengage or leave the platform.

The DS Goal: Build a system to classify students with a high risk of "churn."

Objective 3 – Optimizing Course and Content Effectiveness

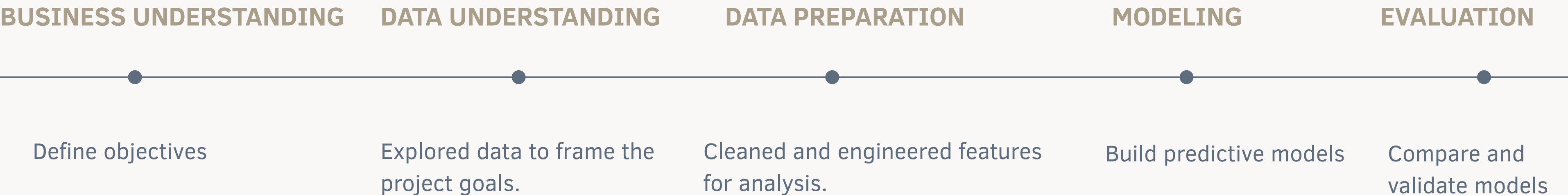
Business Perspective: Understanding which learning materials truly work and which don't.

The DS Goal: Recommend the "next best" choice.

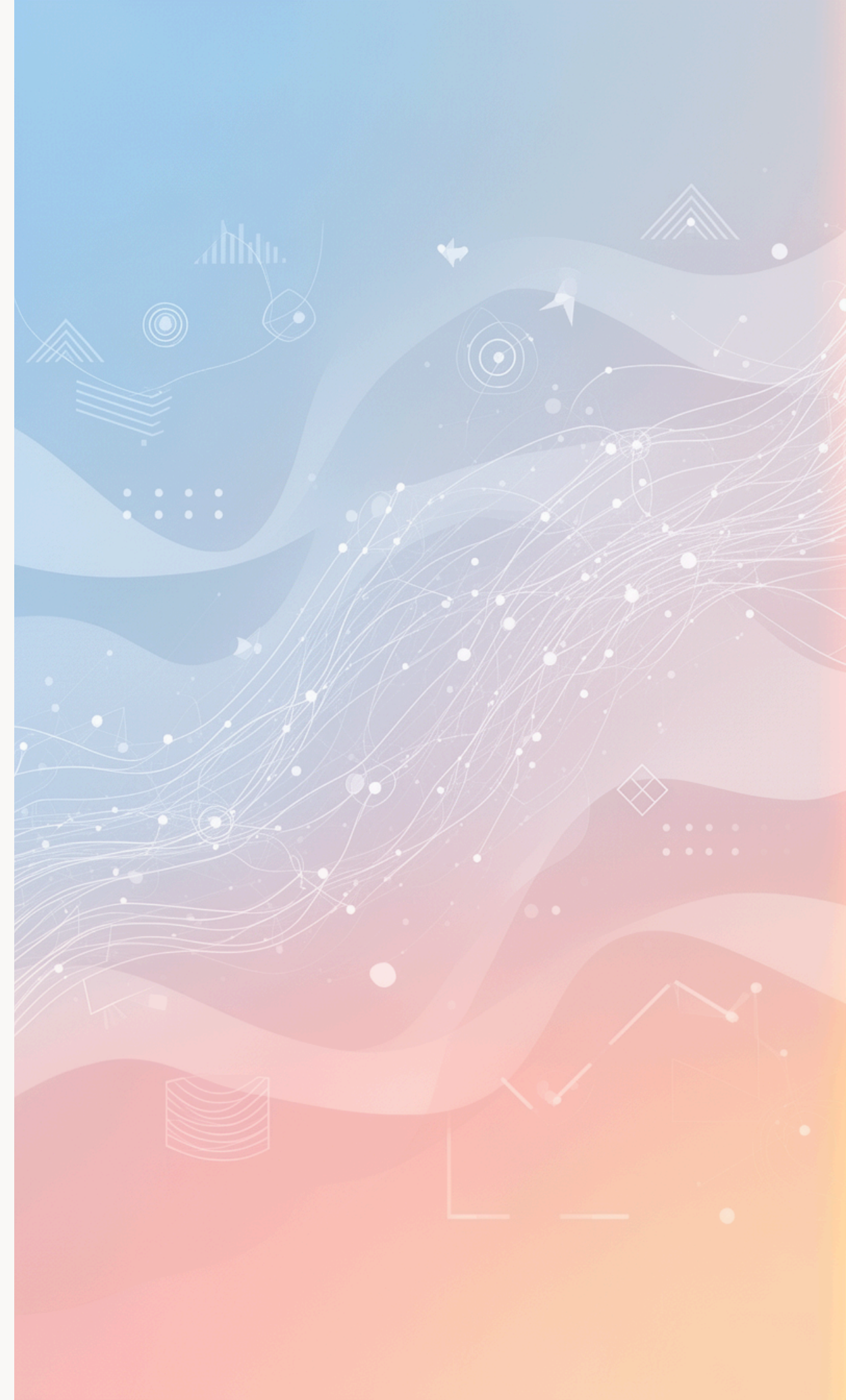
Methodology

We use the CRISP-DM (**Cross-Industry Standard Process for Data Mining**) methodology, which is a standard framework in data science.

It helps structure the project from understanding the business problem to building and evaluating models.



Phase 1 :Understanding Data and Preparing for Analysis



STEP 1 — DATA LOADING

```
from google.colab import files
uploaded = files.upload() # Uploader le fichier cleaned_elearning_dataset (3).csv
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Charger le dataset
df = pd.read_csv('cleaned_elearning_dataset (3).csv')

print(f'Lignes : {df.shape[0]} | Colonnes : {df.shape[1]}')
df.head()
```

Lignes : 10000 | Colonnes : 11

	userid	coursename	sessionduration	sessionsperweek	coursecompletion	usersatisfaction	signupdate	lastactivedate	quizscores	feedbackcomments	activity_duration_days
0	1	team-oriented tangible array	47	8	0	3	2023-12-07	2024-04-27	86.093897	concern near range direction message window.	142
1	2	proactive demand-driven open system	36	1	0	3	2023-04-02	2024-05-17	52.788174	card paper able debate.	411
2	3	re-contextualized encompassing project	71	4	1	3	2023-11-29	2024-03-28	94.891291	treatment difference thank.	120
3	4	right-sized object-oriented process improvement	73	4	0	4	2023-07-07	2024-05-18	85.986530	long its benefit fall from evidence body.	316
4	5	team-oriented heuristic graphic interface	36	1	1	4	2023-06-12	2024-01-16	66.933203	central stage grow senior.	218

Step 2 — Data exploration (EDA)

```
print('=' * 50)
print('SHAPE :', df.shape)
print('=' * 50)
df.info()

=====
SHAPE : (10000, 11)
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   userid                10000 non-null  int64  
 1   coursename            10000 non-null  object  
 2   sessionduration       10000 non-null  int64  
 3   sessionsperweek       10000 non-null  int64  
 4   coursecompletion      10000 non-null  int64  
 5   usersatisfaction      10000 non-null  int64  
 6   signupdate            10000 non-null  object  
 7   lastactivedate        10000 non-null  object  
 8   quizscores            10000 non-null  float64 
 9   feedbackcomments     10000 non-null  object  
10   activity_duration_days 10000 non-null  int64  
dtypes: float64(1), int64(6), object(4)
memory usage: 859.5+ KB
```

```
df.describe().round(2)
```

	userid	sessionduration	sessionsperweek	coursecompletion	usersatisfaction	quizscores	activity_duration_days
count	10000.00	10000.00	10000.00	10000.00	10000.00	10000.00	10000.00
mean	5000.50	62.98	4.95	0.60	3.19	74.80	135.72
std	2886.90	33.64	2.56	0.49	1.30	14.43	106.52
min	1.00	5.00	1.00	0.00	1.00	50.03	1.00
25%	2500.75	33.00	3.00	0.00	2.00	61.88	59.00
50%	5000.50	64.00	5.00	1.00	3.00	74.42	116.00
75%	7500.25	92.00	7.00	1.00	4.00	87.55	167.00
max	10000.00	119.00	9.00	1.00	5.00	100.00	529.00

```
missing = df.isnull().sum()
print('Valeurs manquantes :')
if missing.sum() == 0:
    print('Aucune valeur manquante !')
else:
    print(missing[missing > 0])

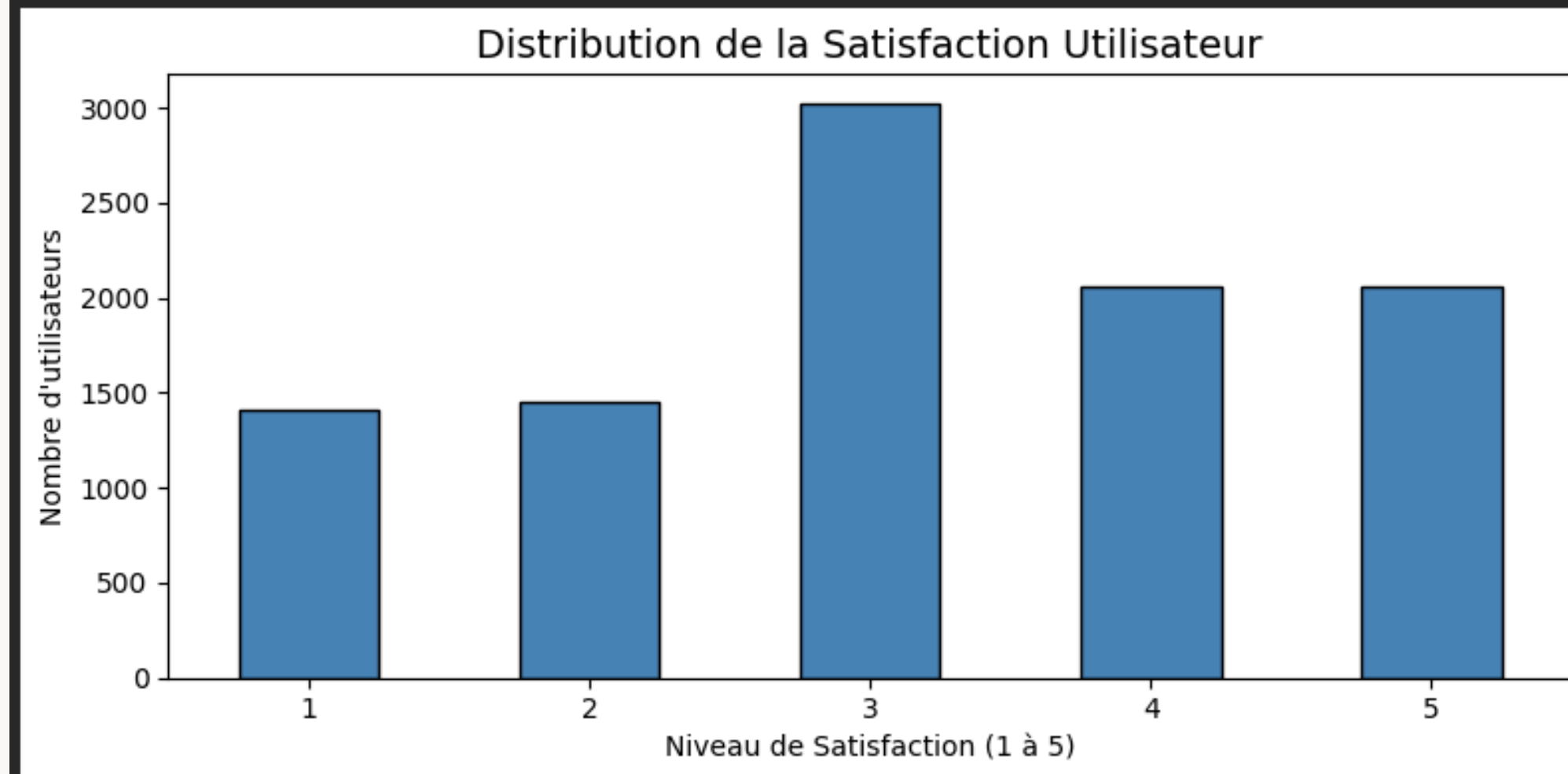
Valeurs manquantes :
Aucune valeur manquante !

duplicates = df.duplicated().sum()
print(f'Nombre de doublons : {duplicates}')
if duplicates > 0:
    df.drop_duplicates(inplace=True)
    print('Doublons supprimés.')

Nombre de doublons : 0
```

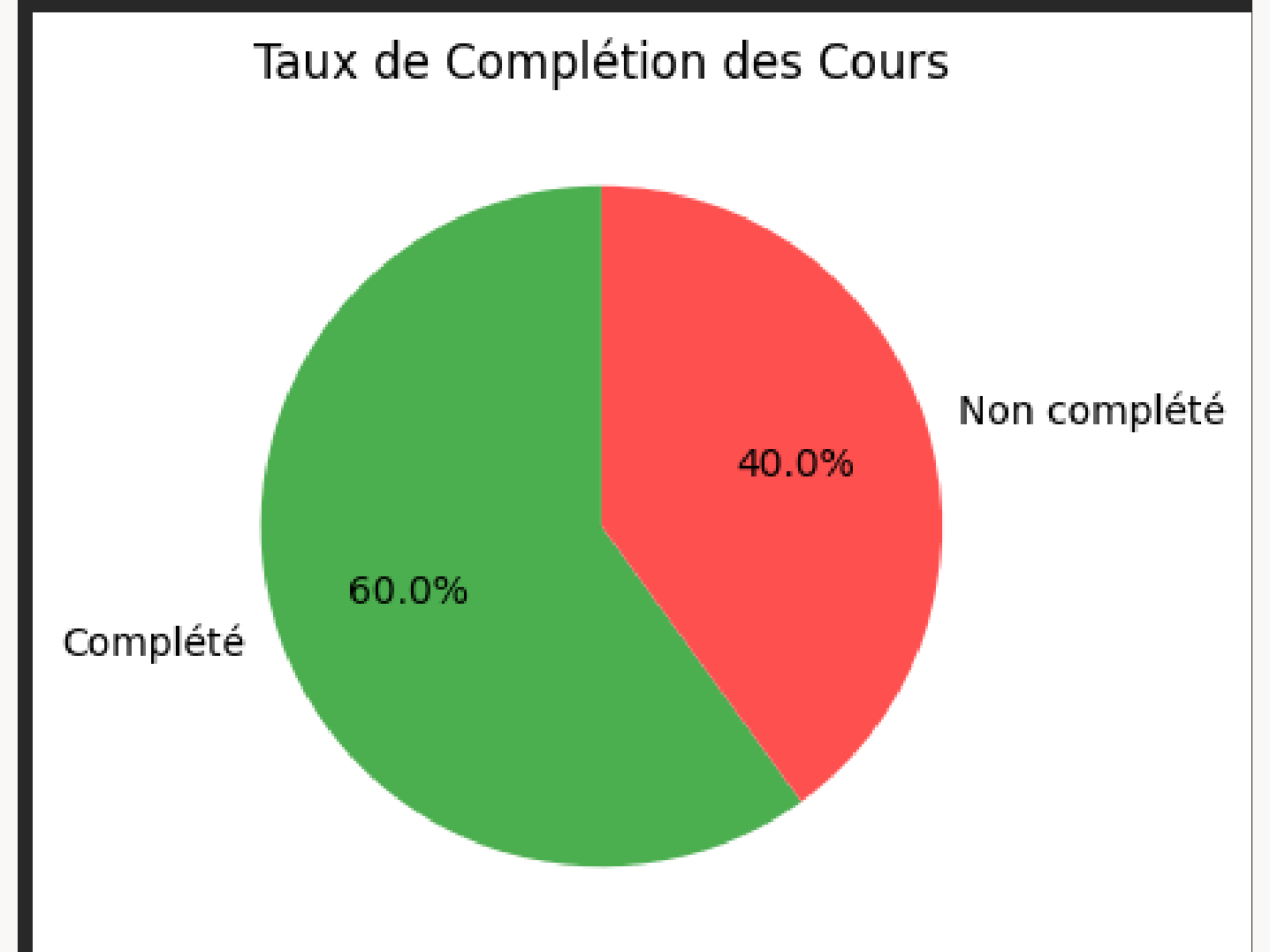
User Satisfaction Distribution

```
plt.figure(figsize=(8, 4))
df['usersatisfaction'].value_counts().sort_index().plot(
    kind='bar', color='steelblue', edgecolor='black')
plt.title('Distribution de la Satisfaction Utilisateur', fontsize=14)
plt.xlabel('Niveau de Satisfaction (1 à 5)')
plt.ylabel("Nombre d'utilisateurs")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Course Completion Rate

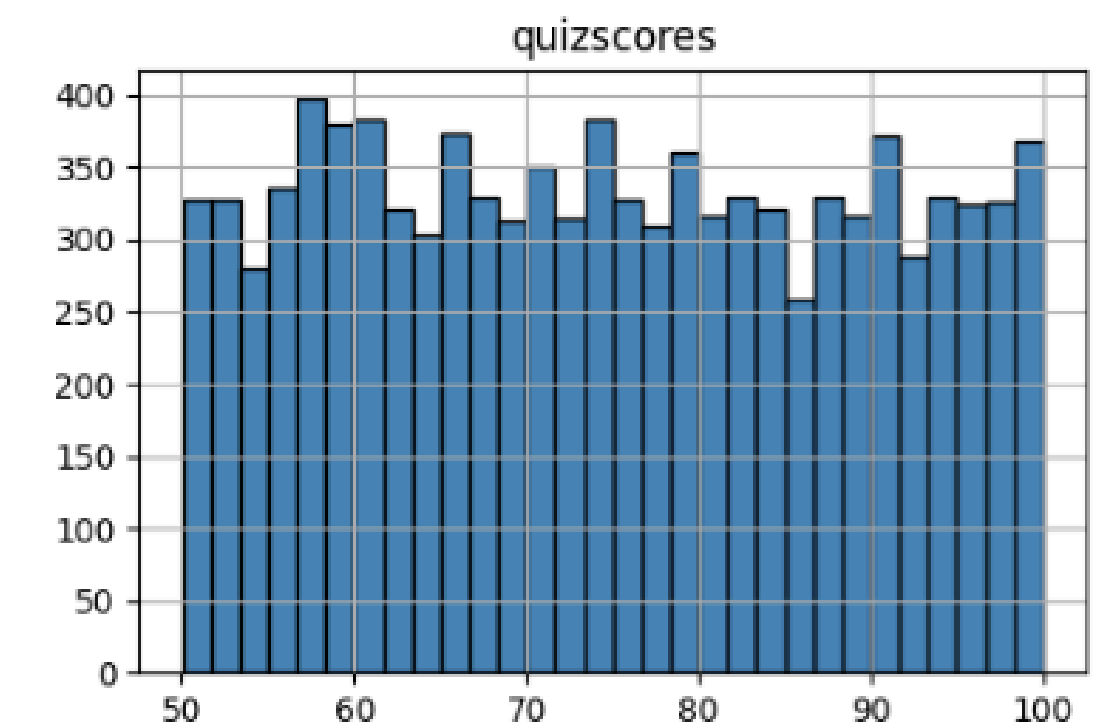
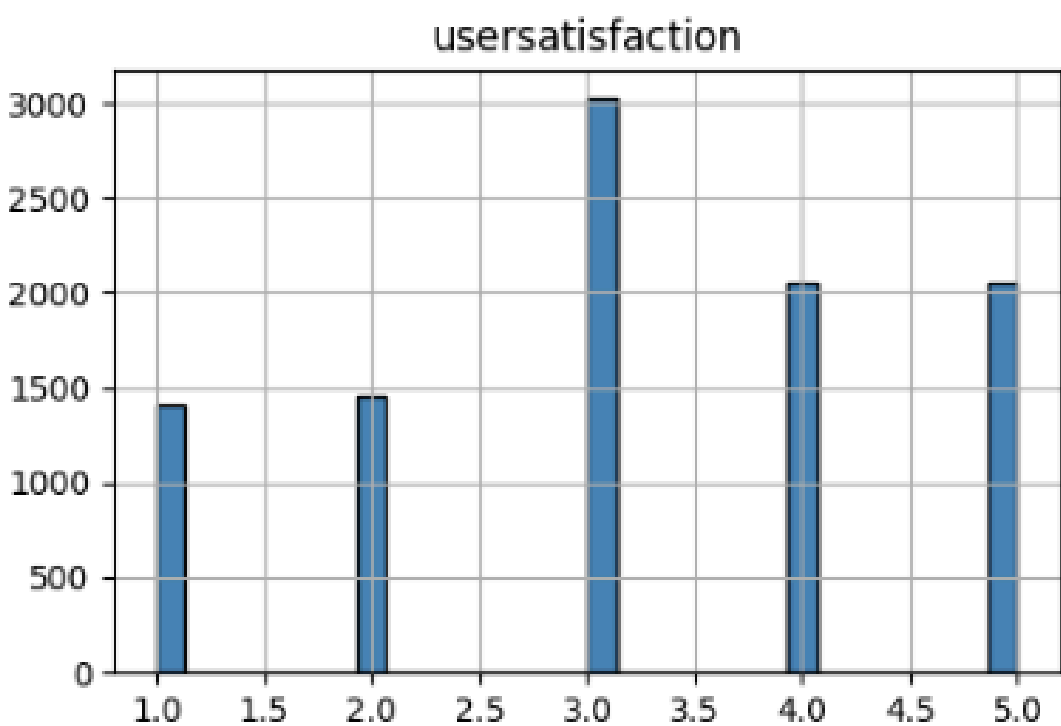
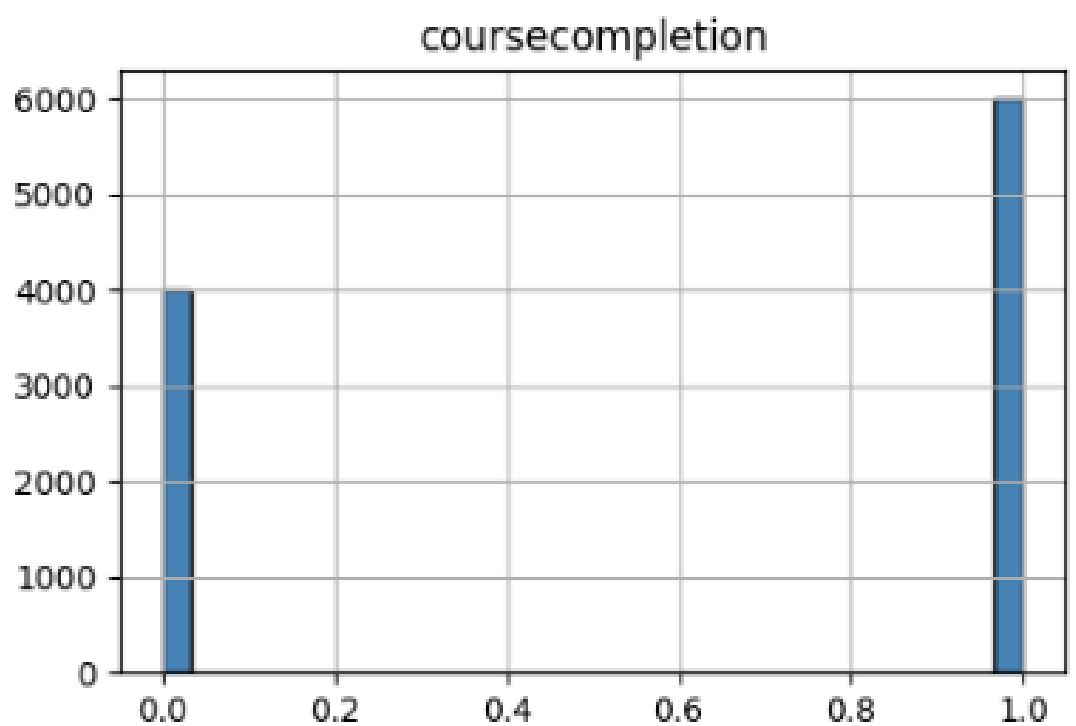
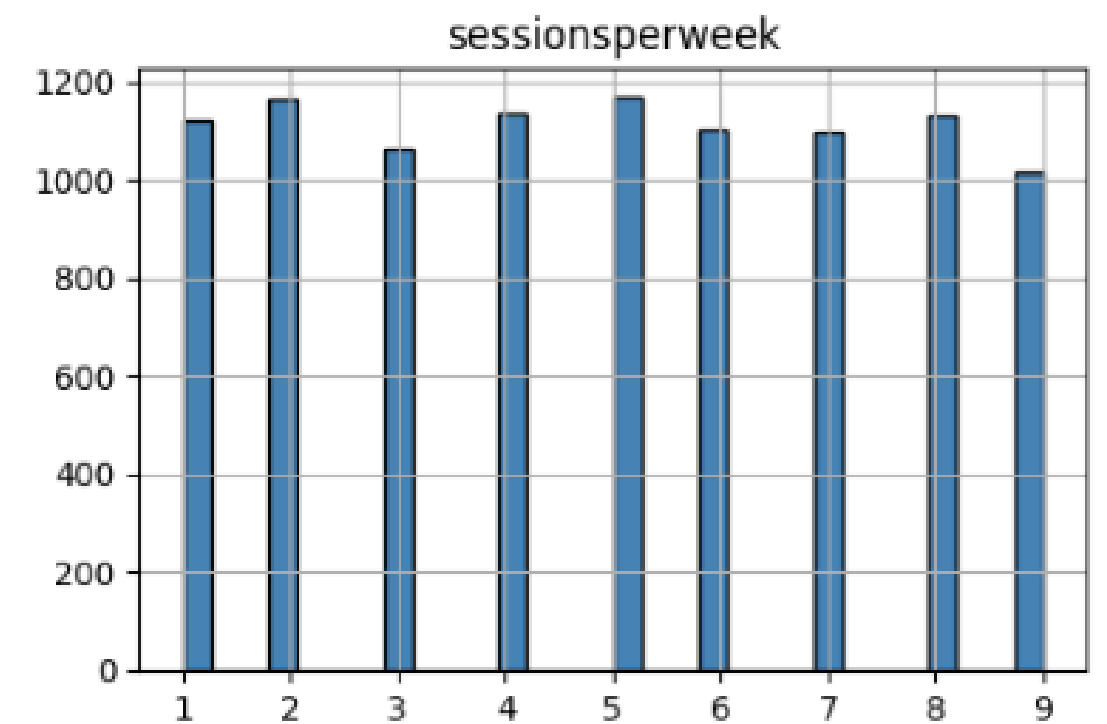
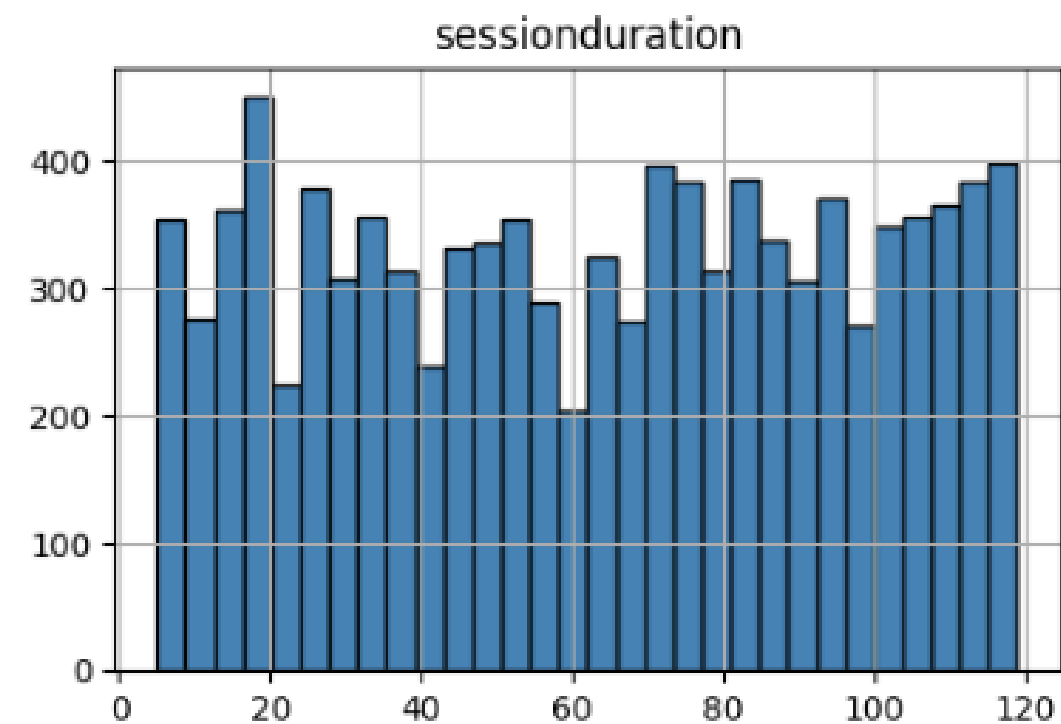
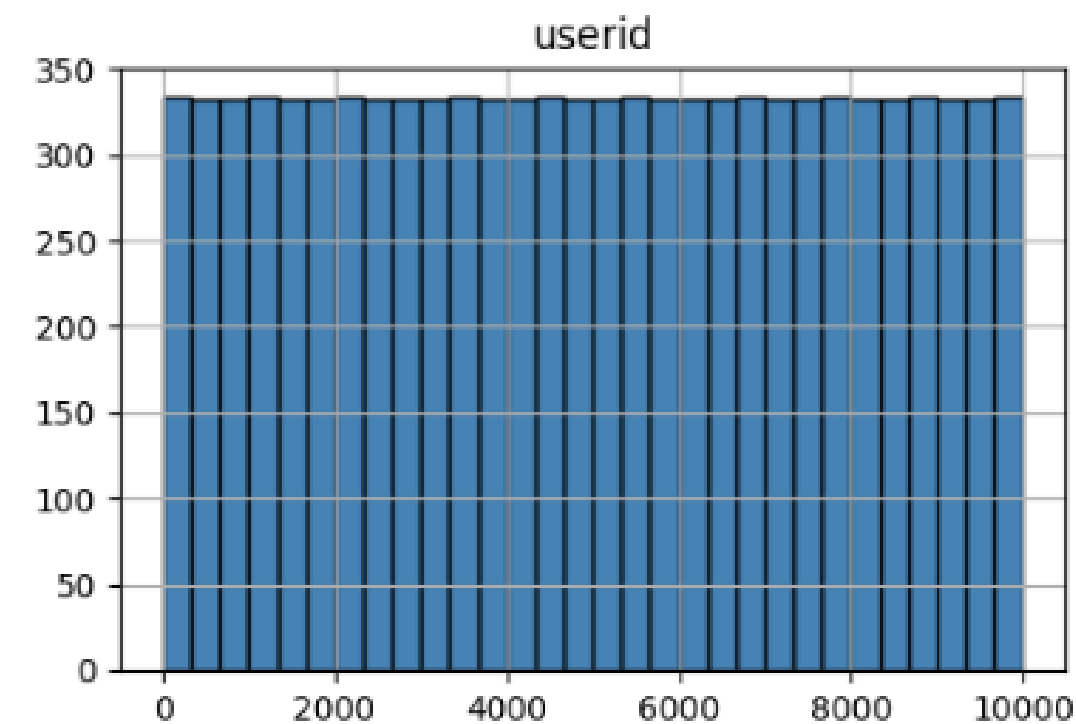
```
plt.figure(figsize=(6, 4))
df['coursecompletion'].value_counts().plot(
    kind='pie', autopct='%1.1f%%',
    labels=['Complété', 'Non complété'],
    colors=['#4CAF50', '#FF5252'], startangle=90)
plt.title('Taux de Complétion des Cours')
plt.ylabel('')
plt.show()
```



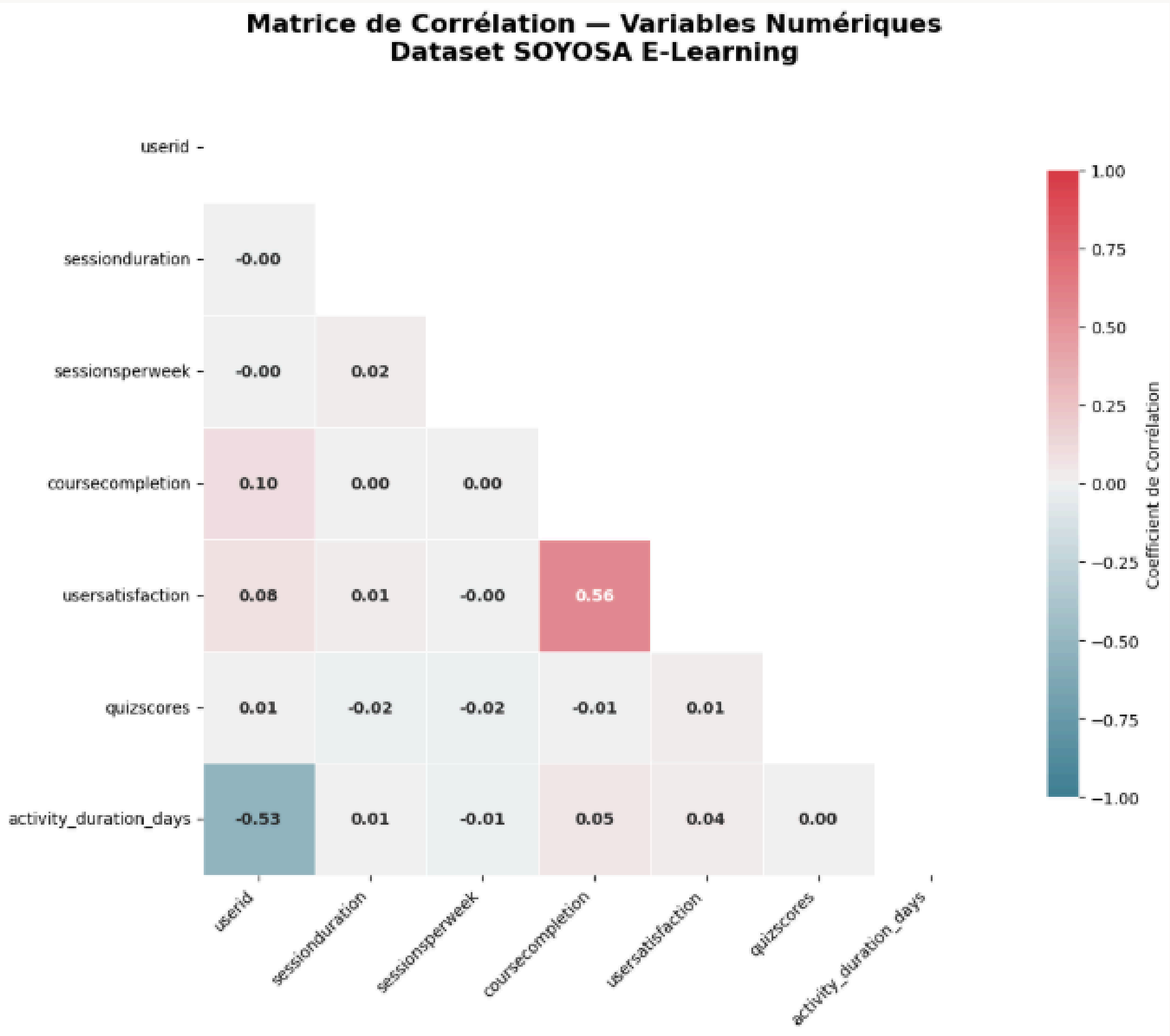
Distribution of Numerical Features

```
num_cols = df.select_dtypes(include=['number']).columns
df[num_cols].hist(figsize=(14, 10), bins=30, edgecolor='black', color='steelblue')
plt.suptitle('Distribution des Variables Numériques', fontsize=16, y=1.02)
plt.tight_layout()
plt.show()
```

Distribution des Variables Numériques



Correlation Matrix Heatmap



 **Corrélations avec usersatisfaction (triées) :**

coursecompletion	+0.561	
userid	+0.082	
activity_duration_days	+0.037	
quizscores	+0.008	
sessionduration	+0.008	
sessionsperweek	-0.002	

Feature la plus corrélée : coursecompletion (0.561)

Step 3 — Data cleaning

```
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(' ', '_')  
    .str.replace(r'^\w', '', regex=True)  
)  
print('Colonnes :', list(df.columns))
```

```
Colonnes : ['userid', 'coursename', 'sessionduration', 'sessionsperweek', 'coursecompletion', 'usersatisfaction', 'signupdate', 'lastactivatedate', 'quizscores', 'feedbackcomments', 'activity_duration_days']
```

```
df['signupdate'] = pd.to_datetime(df['signupdate'])  
df['lastactivatedate'] = pd.to_datetime(df['lastactivatedate'])  
print('Dates converties en datetime')
```

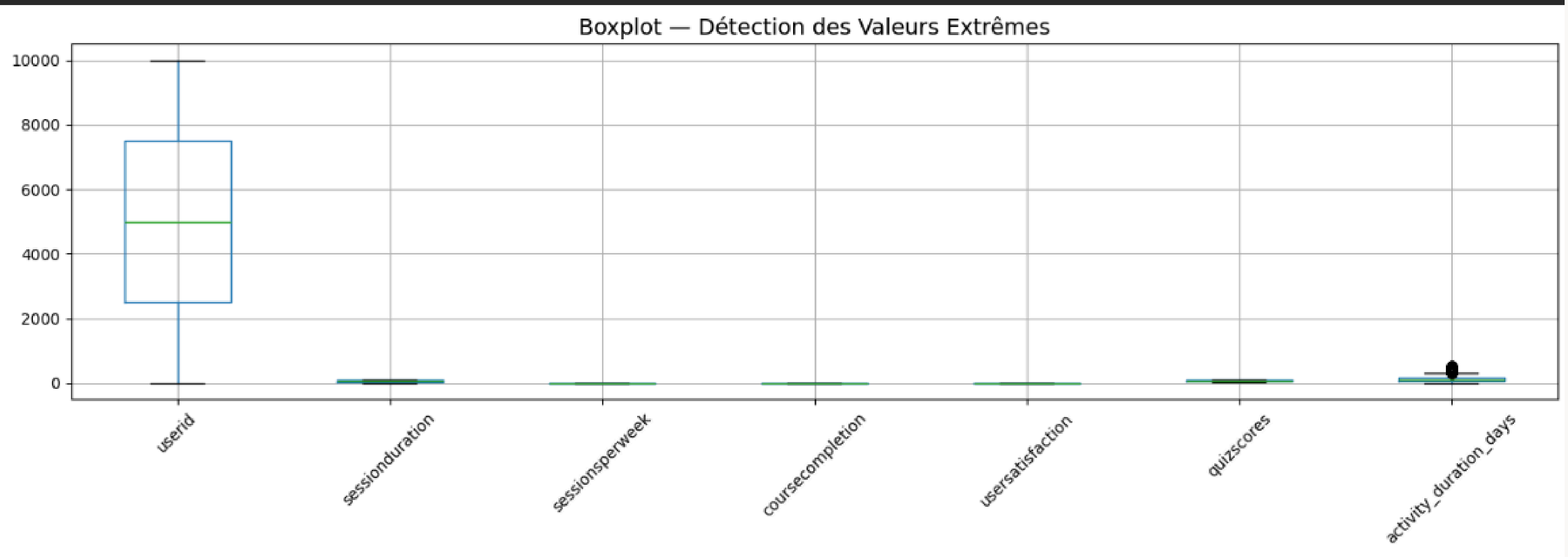
```
Dates converties en datetime
```

```
for col in df.select_dtypes(include=['number']).columns:  
    neg = (df[col] < 0).sum()  
    if neg > 0:  
        print(f'Valeurs négatives dans {col} : {neg}')  
print('Vérification terminée.')
```

```
Vérification terminée.
```

Outlier Detection using Boxplot

```
plt.figure(figsize=(14, 5))
df[num_cols].boxplot()
plt.title('Boxplot - Détection des Valeurs Extrêmes', fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



STEP 4 — FEATURE ENGINEERING

```
df['signup_year']      = df['signupdate'].dt.year
df['signup_month']     = df['signupdate'].dt.month
df['lastactive_year']  = df['lastactivedate'].dt.year
df['lastactive_month'] = df['lastactivedate'].dt.month
df['activity_duration_days'] = (df['lastactivedate'] - df['signupdate']).dt.days
df['feedback_length'] = df['feedbackcomments'].astype(str).apply(len)

print('Nouvelles features créées !')
print(df[['signup_year', 'signup_month', 'lastactive_month', 'feedback_length']].head(3))
```

Nouvelles features créées !

	signup_year	signup_month	lastactive_month	feedback_length
0	2023	12	4	44
1	2023	4	5	23
2	2023	11	3	27



PHASE 2 : MODELING

STEP 5 — DATA LOADING

Defining Features and Target

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
X = df.drop('usersatisfaction', axis=1)
y = df['usersatisfaction']
```

```
print(f'Features (X) : {X.shape[1]} colonnes')
print(f'Cible (y) : valeurs = {sorted(y.unique())}')

```

```
Features (X) : 29 colonnes
Cible (y) : valeurs = [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]

```

Train-Test Split & Feature Scaling

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(f'Train : {X_train.shape[0]} lignes | Test : {X_test.shape[0]} lignes')
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print('Données normalisées (StandardScaler)')
```

```
Train : 8000 lignes | Test : 2000 lignes
Données normalisées (StandardScaler)

```

Linear Regression: Training & Evaluation

```
lin_model = LinearRegression()
lin_model.fit(X_train_scaled, y_train)
y_pred_lin = lin_model.predict(X_test_scaled)

mae_lin = mean_absolute_error(y_test, y_pred_lin)
mse_lin = mean_squared_error(y_test, y_pred_lin)
r2_lin = r2_score(y_test, y_pred_lin)

print('Linear Regression :')
print(f' MAE : {mae_lin:.4f}')
print(f' MSE : {mse_lin:.4f}')
print(f' R2 : {r2_lin:.4f}')
```

Linear Regression :
MAE : 0.8920
MSE : 1.1723
R2 : 0.3007

```
#Evaluation
Accuracy = accuracy_score(y_test, y_pred)
confm = confusion_matrix(y_test, y_pred)
classr = classification_report(y_test, y_pred)
print("Accuracy", Accuracy)
print("Confusion Matrix", confm)
print("Classification Report", classr)
```

Accuracy 0.899
Confusion Matrix [[269 64]
[37 630]]
Classification Report

		precision	recall	f1-score	support
0	0.88	0.81	0.84	333	
1	0.91	0.94	0.93	667	
accuracy		0.90	1000		
macro avg	0.89	0.88	0.88	1000	
weighted avg	0.90	0.90	0.90	1000	

LogisticRegression

Random Forest: Training & Evaluation

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print('Random Forest :')
print(f' MAE : {mae_rf:.4f}')
print(f' MSE : {mse_rf:.4f}')
print(f' R2 : {r2_rf:.4f}')
```

Random Forest :
MAE : 0.8873
MSE : 1.1403
R2 : 0.3198

```
cv_lin = cross_val_score(LinearRegression(), X_train_scaled, y_train, cv=5, scoring='r2')
cv_rf = cross_val_score(RandomForestRegressor(n_estimators=100, random_state=42),
| | | | | X_train, y_train, cv=5, scoring='r2')

print('Cross-Validation R2 (moyenne +/- écart-type) :')
print(f' Linear Regression : {cv_lin.mean():.4f} +/- {cv_lin.std():.4f}')
print(f' Random Forest : {cv_rf.mean():.4f} +/- {cv_rf.std():.4f}')
```

Cross-Validation R2 (moyenne +/- écart-type) :
Linear Regression : 0.3118 +/- 0.0239
Random Forest : 0.3410 +/- 0.0180

Cross-Validation

STEP 6 — MODEL COMPARISON

Model Performance Summary

```
results_df = pd.DataFrame({
    'Modèle': ['Linear Regression', 'Random Forest'],
    'MAE': [round(mae_lin,4), round(mae_rf,4)],
    'MSE': [round(mse_lin,4), round(mse_rf,4)],
    'R2': [round(r2_lin,4), round(r2_rf,4)],
    'CV R2 moyen': [round(cv_lin.mean(),4), round(cv_rf.mean(),4)]
})
print(results_df.to_string(index=False))
```

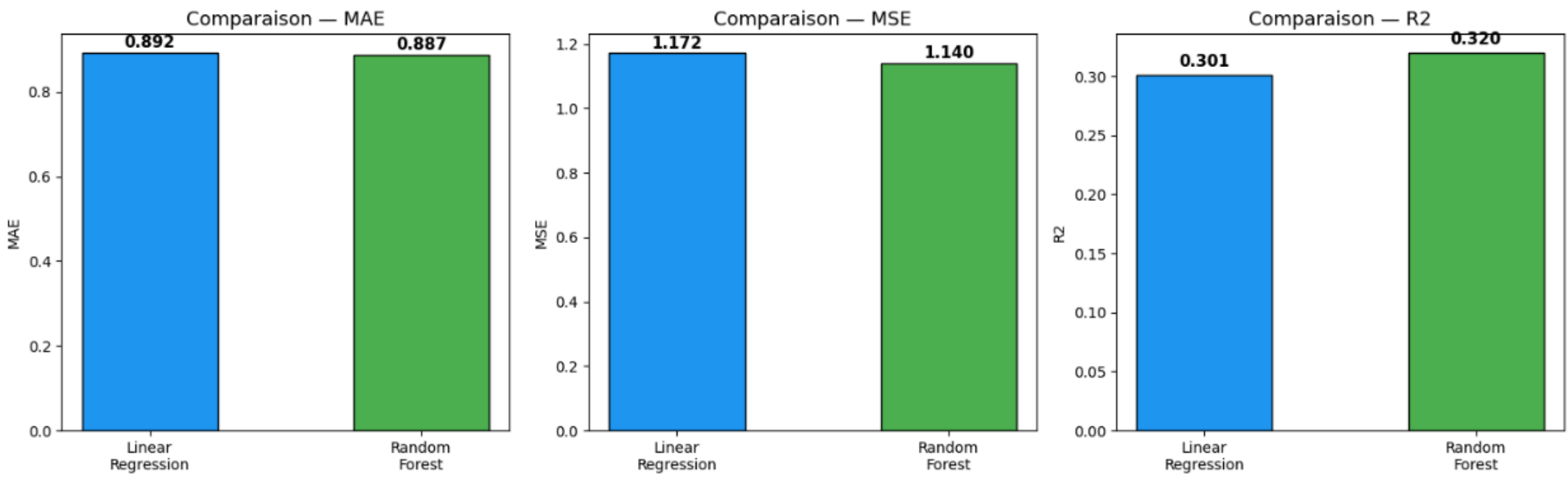
Modèle	MAE	MSE	R2	CV R2 moyen
Linear Regression	0.8920	1.1723	0.3007	0.3118
Random Forest	0.8873	1.1403	0.3198	0.3410

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
models = ['Linear\nRegression', 'Random\nForest']
colors = ['#2196F3', '#4CAF50']

for ax, metric, vals in zip(
    axes,
    ['MAE', 'MSE', 'R2'],
    [[mae_lin, mae_rf], [mse_lin, mse_rf], [r2_lin, r2_rf]]):
    bars = ax.bar(models, vals, color=colors, edgecolor='black', width=0.5)
    ax.set_title(f'Comparaison - {metric}', fontsize=13)
    ax.set_ylabel(metric)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2,
                bar.get_height() + 0.005,
                f'{val:.3f}', ha='center', va='bottom',
                fontsize=11, fontweight='bold')

plt.suptitle('Comparaison : Linear Regression vs Random Forest', fontsize=14)
plt.tight_layout()
plt.show()
```

Comparaison : Linear Regression vs Random Forest



Model Comparison Visualization

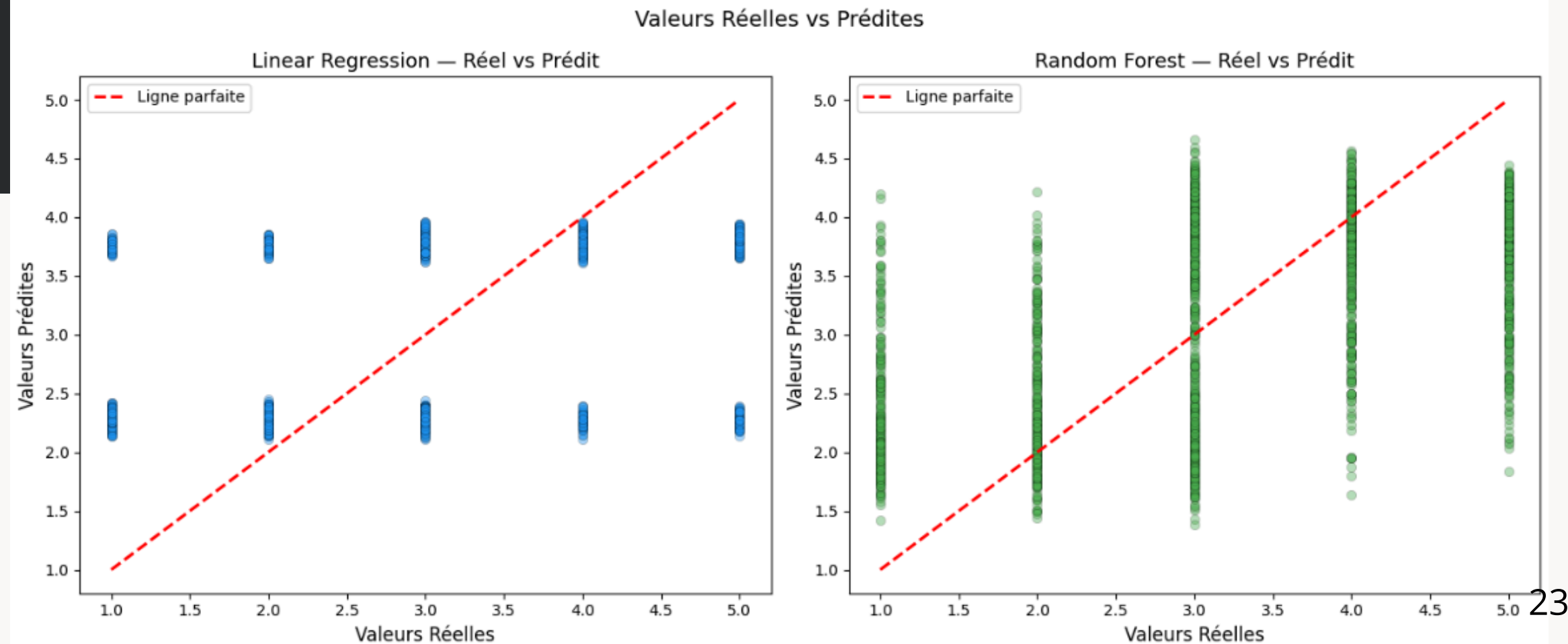
STEP 7 — PREDICTION VISUALIZATION

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for ax, y_pred, title, color in zip(
    axes,
    [y_pred_lin, y_pred_rf],
    ['Linear Regression', 'Random Forest'],
    ['#2196F3', '#4CAF50']):
    ax.scatter(y_test, y_pred, alpha=0.4, color=color,
               edgecolors='black', linewidths=0.3)
    ax.plot([y_test.min(), y_test.max()],
            [y_test.min(), y_test.max()],
            color='red', linewidth=2, linestyle='--', label='Ligne parfaite')
    ax.set_xlabel('Valeurs Réelles', fontsize=12)
    ax.set_ylabel('Valeurs Prédites', fontsize=12)
    ax.set_title(f'{title} — Réel vs Prédit', fontsize=13)
    ax.legend()

plt.suptitle('Valeurs Réelles vs Prédites', fontsize=14)
plt.tight_layout()
plt.show()
```

Model Prediction Implementation

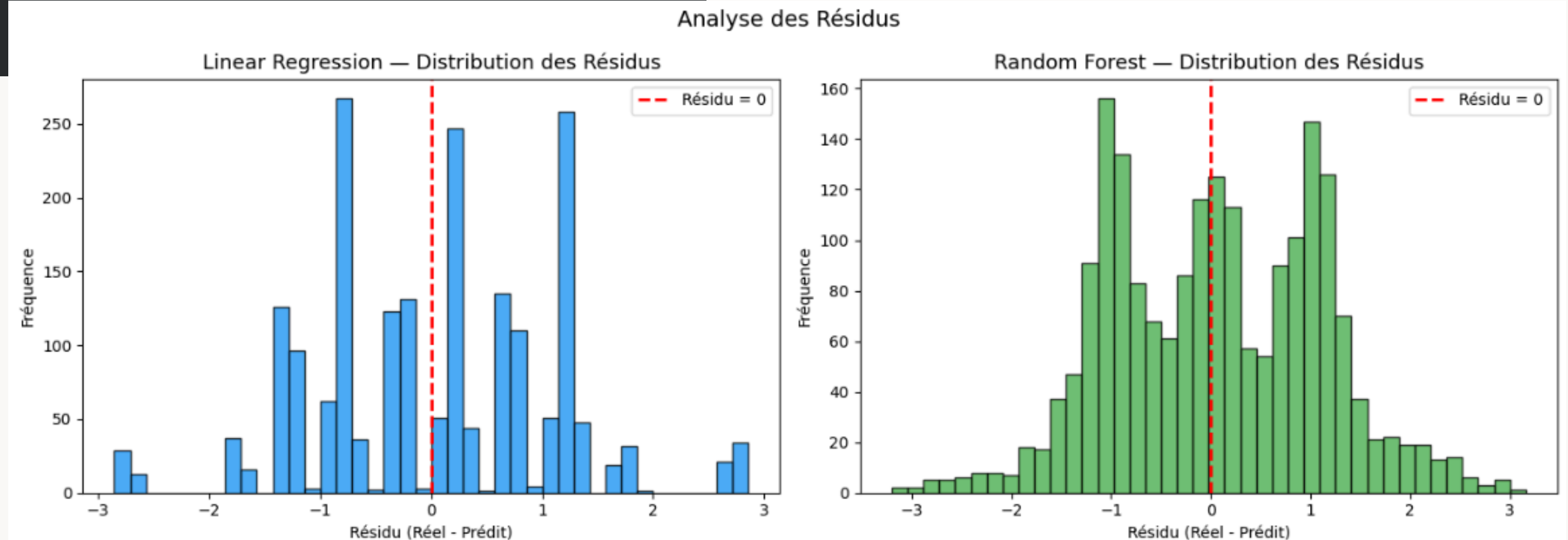


```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, y_pred, title, color in zip(
    axes,
    [y_pred_lin, y_pred_rf],
    ['Linear Regression', 'Random Forest'],
    ['#2196F3', '#4CAF50']):
    residuals = y_test - y_pred
    ax.hist(residuals, bins=40, color=color, edgecolor='black', alpha=0.8)
    ax.axvline(0, color='red', linewidth=2, linestyle='--', label='Résidu = 0')
    ax.set_title(f'{title} — Distribution des Résidus', fontsize=13)
    ax.set_xlabel('Résidu (Réel - Prédit)')
    ax.set_ylabel('Fréquence')
    ax.legend()

plt.suptitle('Analyse des Résidus', fontsize=14)
plt.tight_layout()
plt.show()
```

Model Residual Analysis



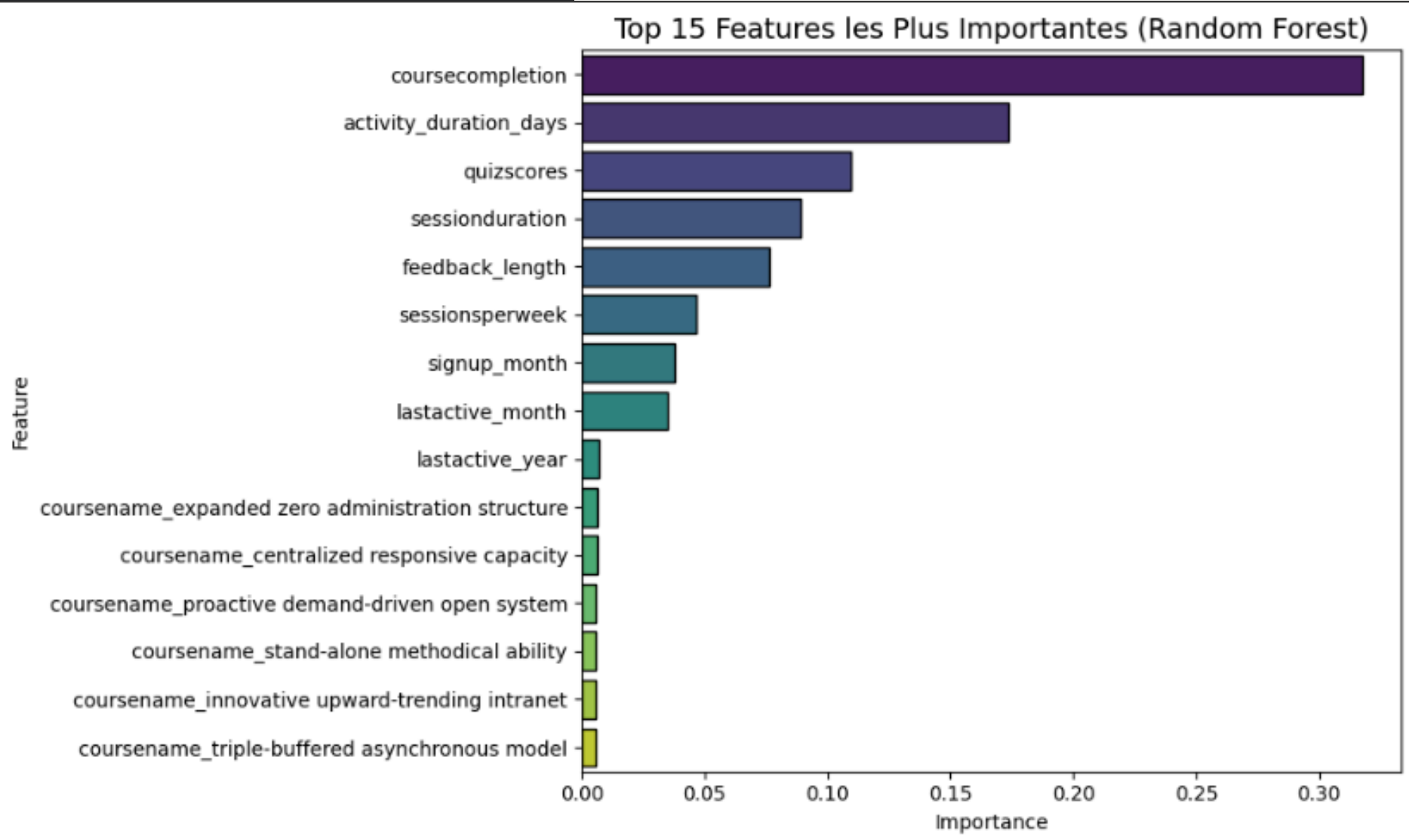
STEP 8 — FEATURE IMPORTANCE

```
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False).head(15)

plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df, x='Importance', y='Feature',
            palette='viridis', edgecolor='black')
plt.title('Top 15 Features les Plus Importantes (Random Forest)', fontsize=14)
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

print(importance_df.head(10).to_string(index=False))
```

Feature	Importance
coursecompletion	0.317378
activity_duration_days	0.173717
quizscores	0.109294
sessionduration	0.088962
feedback_length	0.076478
sessionsperweek	0.046775
signup_month	0.037886
lastactive_month	0.035103
lastactive_year	0.006889
coursename_expanded zero administration structure	0.006308



Essential Tools and Technologies Used

Programming

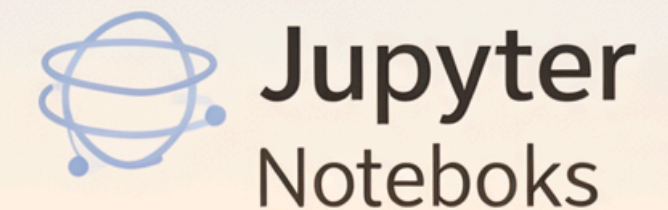
- Python

Machine Learning & Data

- Pandas
- NumPy
- Scikit-learn

Environment

- Jupyter Notebooks



Conclusion

In this project, we developed and evaluated multiple Machine Learning models to predict user satisfaction based on learning platform data.

The results show that the Random Forest model outperforms Linear Regression, achieving better performance .

This confirms the importance of using more advanced models to capture complex and non-linear relationships in the data.

Overall, our approach demonstrates how data-driven techniques can significantly improve understanding and decision-making in e-learning platforms.

Next Steps

- Develop a classification model to predict course completion
- Improve model performance through optimization and tuning
- Integrate the model into an API (Flask / FastAPI)
- Connect the solution to a real learning platform

Thank you
for your
attention!

